PYTHON FOUNDATIONS · 40 MIN

GUI Programming: Python Development Options

tkinter
Built into Python
`import tkinter`

Qt
PySide6 and PyQt6
`QApplication`

Kivy
Touch and mobile
`kivy.app`

Web
Interfaces in a browser
`Streamlit Dash`

~3 min

Four Parts of GUI Programming

From the idea of a window to choosing a toolkit.

1

Concepts

Windows, widgets and the event loop.

mainloop

2

Toolkits

The main ways to build a GUI in Python.

tkinter Qt

3

tkinter

Widgets, layout and callbacks in practice.

pack grid

4

Choosing

Match the toolkit to the project.

Web Kivy

> A graphical user interface lets people act with a mouse and keyboard instead of typed commands.

~3 min

From Command Line to GUI

Same program ideas, a different way to talk to the user.

Feature	Command line	GUI
Input	Typed text and arguments	Clicks, keys and text fields
Flow	The program chooses the next step	User actions choose the next step
Output	Printed lines	Windows, labels and dialogs
Best for	Scripts and automation	Interactive tools

A GUI program waits for events and reacts to them, instead of running straight to the end.

~4 min

The Event Loop

Every GUI toolkit works this way.

Build widgets
Create the window and its controls.

→

Start the loop
mainloop() begins waiting.

→

Wait for events
Clicks, keys and timers arrive.

→

Run a callback
Your function handles the event.

Event-driven programming
A GUI program builds its window, then hands control to the toolkit. Each user action calls a function you registered earlier, known as a callback, and then the loop goes back to waiting.

~4 min

Python GUI Options

The main toolkits at a glance.

Option	Kind	Notes
tkinter	Standard library	Ships with most Python installs, simple
PyQt6 and PySide6	Qt bindings	Rich widgets, and a visual designer
wxPython	wxWidgets binding	Widgets with a native look
Kivy	Own toolkit	Touch screens and mobile apps
Web frameworks	Browser interface	Streamlit, Dash and Flask

tkinter comes with most Python installs, and the other options are installed with pip.

~4 min

A First tkinter Window

Four lines build, then mainloop shows it.

```
import tkinter as tk
root = tk.Tk()
root.title("Hello")
tk.Label(root, text="Hi").pack()
root.mainloop()
```

Result

```
A window titled Hello shows Hi
```

Tk and mainloop

Tk() creates the main window. mainloop() starts the event loop and waits.

pack()

Places the widget in its window. A widget with no layout call never appears.

~4 min

Common tkinter Widgets

Building blocks of a window.

Widget	Purpose	Example
Label	Shows text or an image	tk.Label(root, text='Name')
Button	Runs a function when clicked	tk.Button(root, text='OK')
Entry	One-line text input	tk.Entry(root)
Text	Multi-line text area	tk.Text(root)
Listbox	A list the user can pick from	tk.Listbox(root)

> Every widget takes its parent window as the first argument.

~4 min

Layout Managers

Three ways to place widgets in a window.

1

pack

Stacks widgets top to bottom or side by side.

w.pack()

2

grid

Places widgets in rows and columns, like a table.

w.grid(row=0, column=1)

3

place

Puts a widget at exact x and y positions.

w.place(x=10, y=20)

> Do not mix pack and grid in the same container. Tk reports an error if you do.

~4 min

Qt With PySide6

A larger toolkit with the same ideas.

```
from PySide6.QtWidgets import (
    QApplication, QLabel)
app = QApplication([])
label = QLabel("Hello from Qt")
label.show()
app.exec()
```

Result

```
A window shows Hello from Qt
```

PySide6 and PyQt6

Python bindings for the Qt toolkit. Install with pip, and the code looks almost the same in both.

A rich toolkit

Qt offers many widgets, layouts and styling options, plus a visual layout tool.

~4 min

Web-Based Interfaces

Let the browser be the window.

```
import streamlit as st
name = st.text_input("Name")
if name:
    st.write("Hello,", name)
```

Result

A web page with a Name box that greets you

Browser as the window

Streamlit, Dash and Flask draw the interface in a web page instead of a desktop window.

Run and share

A Streamlit app starts with streamlit run app.py, and others can open it by URL.

~3 min

Choosing an Option

Match the toolkit to the project.

Need	Pick
A quick utility with nothing to install	tkinter
A polished desktop application	PySide6 or PyQt6
A native look on each platform	wxPython
A touch or mobile app	Kivy
A dashboard or data tool	Streamlit or Dash

> To learn GUI ideas, tkinter is the easiest start, because it comes with Python.

ONE-LINE TAKEAWAY

A GUI program builds widgets, then lets an event loop call your functions when the user acts.

widgets layout events mainloop

PYTHON FOUNDATIONS · 45 MIN

Adding Widgets in tkinter

Widgets
Labels and buttons

`Label` `Button`

Containers
Group and arrange

`Frame`

Input
Collect user data

`Entry` `Scale`

Lists
Choose from items

`Listbox` `Menu`

~3 min

Four Steps in Building a Window

Add widgets one at a time, then arrange them.

1

Add

Create widgets with a parent window.

tk.Label(root)

2

Arrange

Place them with pack and frames.

pack Frame

3

Collect

Read what the user types or picks.

Entry Checkbutton

4

Extend

Add lists, drawings, menus and dialogs.

Listbox Menu

A tkinter window is a tree of widgets, and each one is added to a parent.

~4 min

Adding Your First Widgets

Create each widget, then lay it out.

```
import tkinter as tk
root = tk.Tk()
msg = tk.Label(root, text="Hello")
msg.pack()
btn = tk.Button(root, text="Quit",
                command=root.destroy)
btn.pack()
root.mainloop()
```

Result

```
A window with Hello above a Quit button
```

Parent first

Each widget is created with its parent as the first argument, which decides where it lives.

Two steps

Create the widget, then give it a layout call such as pack. Both steps are needed.

~4 min

Common Widget Options

Style a widget when you create it.

Option	Effect	Example
text	The text shown	text='Hi'
font	Typeface and size	font=('Arial', 14)
bg and fg	Background and text colour	bg='white', fg='navy'
width	Width in characters	width=20
state	Normal or disabled	state='disabled'

Options can be passed when the widget is created, or changed later with widget.config().

~4 min

Packing Options

Control where a widget goes and how it grows.

Option	Values	Effect
side	top, bottom, left, right	Which edge to attach to
fill	x, y or both	Stretch to fill the free space
expand	True or False	Claim extra space when the window grows
padx, pady	Pixels	Space around the widget
anchor	n, s, e, w and others	Position inside its space

fill and expand together let a widget grow when the user resizes the window.

~4 min

Grouping With Frames

A Frame holds a row or a block of widgets.

```
top = tk.Frame(root)
top.pack(fill="x")
tk.Label(top, text="Name").pack(side="left")
tk.Entry(top).pack(side="left")
tk.Button(root, text="Save").pack()
```

Result

```
A row with Name and a text box,
and a Save button below it
```

Frame as a container

A Frame groups widgets. Widgets created with the frame as parent are packed inside it.

Rows of widgets

pack(side='left') inside a frame builds a row, and stacked frames make a form.

~5 min

Entry and StringVar

Read what the user types.

```
name = tk.StringVar()
tk.Entry(root, textvariable=name).pack()
def show():
    result.config(text="Hi " + name.get())
tk.Button(root, text="Greet",
          command=show).pack()
result = tk.Label(root)
result.pack()
```

Result

```
Typing Asha and pressing Greet shows Hi Asha
```

StringVar

A variable object linked to a widget. Its get() returns the current text.

entry.get()

An Entry also has its own get() method, so a StringVar is optional.

~4 min

Checkbuttons and Radiobuttons

Choose on or off, or one of several.

```
agree = tk.BooleanVar()
tk.Checkbutton(root, text="Agree",
    variable=agree).pack()
color = tk.StringVar(value="red")
tk.Radiobutton(root, text="Red",
    value="red", variable=color).pack()
tk.Radiobutton(root, text="Blue",
    value="blue", variable=color).pack()
```

Result

```
A check box, and two radio buttons
that share one choice
```

Checkbutton

Tracks on or off in a BooleanVar. agree.get() returns True or False.

Radiobutton

Radio buttons that share one variable form a group where only one choice is selected.

~4 min

Listbox and Scrollbar

Show many items, with scrolling.

```
frame = tk.Frame(root)
frame.pack()
lb = tk.Listbox(frame, height=4)
sb = tk.Scrollbar(frame, command=lb.yview)
lb.config(yscrollcommand=sb.set)
lb.insert("end", "a", "b", "c", "d", "e")
lb.pack(side="left")
sb.pack(side="right", fill="y")
```

Result

A list with a scrollbar beside it

Two-way link

The scrollbar's command moves the list, and the list's yscrollcommand updates the scrollbar.

curselection()

lb.curselection() returns the positions of the selected items.

~4 min

Scale and Spinbox

Pick a number with a slider or a box.

```
v = tk.IntVar(value=5)
tk.Scale(root, from_=0, to=10,
         orient="horizontal",
         variable=v).pack()
tk.Spinbox(root, from_=0, to=10).pack()
```

Result

```
A slider and a number box,
both running from 0 to 10
```

from_ and to

Scale and Spinbox take a range. The name is from_ because from is a Python keyword.

Variables update

Linking a variable lets code read or set the slider with v.get() and v.set().

~4 min

Text and Canvas

Multi-line text, and drawing.

```
t = tk.Text(root, height=3, width=20)
t.insert("1.0", "Notes")
t.pack()
c = tk.Canvas(root, width=100, height=60)
c.create_rectangle(10, 10, 90, 50)
c.pack()
```

Result

```
A text box holding Notes, and
a drawing area with a rectangle
```

Text widget

Text holds many lines. Positions are written as line.column, so '1.0' is the very start.

Canvas

Canvas draws shapes, lines and images with methods such as create_rectangle and create_line.

~4 min

Menus

A menu bar across the top of the window.

```
bar = tk.Menu(root)
file = tk.Menu(bar, tearoff=0)
file.add_command(label="Quit",
                 command=root.destroy)
bar.add_cascade(label="File", menu=file)
root.config(menu=bar)
```

Result

```
A File menu with a Quit item
```

Menu and cascade

Create a Menu, add commands to it, then attach it to the bar with add_cascade.

Attach to the window

root.config(menu=bar) shows the bar at the top of the window.

~4 min

Standard Dialogs

Ask or tell the user with a pop-up.

```
from tkinter import messagebox
def ask():
    if messagebox.askyesno("Quit", "Sure?"):
        root.destroy()
b = tk.Button(root, text="Quit",
              command=ask)
b.pack()
```

Result

Clicking Quit opens a Sure? question with Yes and No

messagebox

showinfo, showwarning, showerror, askyesno and askokcancel open standard dialogs.

Return values

Question dialogs return True or False, so you can use them directly in an if.

~3 min

Widget Cheat Sheet

Match the need to the widget.

Need	Widget
Show text	Label
Run an action	Button
Type one line	Entry
Choose on or off	Checkbutton
Choose one of several	Radiobutton

> For many items use Listbox, for long text use Text, and for drawing use Canvas.

ONE-LINE TAKEAWAY

Create each widget with a parent, lay it out, and link variables and commands to react to the user.

Frame pack StringVar command

PYTHON FOUNDATIONS · 50 MIN

GUI Coding Techniques

Classes
GUIs as objects
`class App(Frame)`

Callbacks
Pass data safely
`lambda partial`

Reuse
Build components
`LabeledEntry`

Timing
Timers and threads
`after Queue`

~3 min

Four Techniques for Bigger GUIs

Patterns that keep growing programs manageable.

1

Classes

Wrap a window or component in a class.

class self

2

Callbacks

Pass data to handlers without globals.

partial

3

Components

Build reusable groups of widgets.

Frame

4

Timing

Run timers and long jobs without freezing.

after Queue

Small scripts can use plain functions, but real GUIs stay readable when organised into classes.

~5 min

A Class-Based GUI

Subclass Frame and add widgets inside it.

```
class App(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.pack()
        tk.Label(self, text="Hi").pack()
root = tk.Tk()
App(root)
root.mainloop()
```

Result

A window that shows Hi

Subclass Frame

The class is a Frame, so it can be packed into any window and reused.

master

master is the parent window. Pass it to super().__init__ so the frame attaches to it.

~5 min

Callbacks as Methods

State lives on the object.

```
class Counter(tk.Label):
    def __init__(self, master):
        super().__init__(master, text="0")
        self.n = 0
        self.bind("<Button-1>", self.add)
    def add(self, event):
        self.n += 1
        self.config(text=self.n)
```

Result

```
Clicking the label raises its number
```

Bound methods

self.add carries its object with it, so a callback can use self.n and self.config.

The event argument

Handlers attached with bind() receive an event object. Button commands receive no arguments.

~5 min

Passing Arguments to Callbacks

One handler, many buttons.

```
from functools import partial
def say(word):
    print("Pressed", word)
for w in ["A", "B"]:
    tk.Button(root, text=w,
        command=lambda w=w: say(w)).pack()
tk.Button(root, text="C",
    command=partial(say, 'C')).pack()
```

Result

```
Each button prints its own letter
```

The late-binding trap

A lambda that uses the loop variable directly sees its final value. The default w=w captures the current one.

functools.partial

partial(say, 'C') builds a callback with the argument already filled in.

~4 min

Ways to Pass Data to a Callback

Choose the clearest option for the job.

Technique	Example	Best when
Bound method	command=self.add	State lives on an object
Lambda with default	lambda w=w: say(w)	A quick one-off handler
partial	partial(say, 'C')	The call is simple and reusable
Closure	def make(w): return lambda: say(w)	A factory builds many handlers

Avoid global variables for GUI state, because they are hard to follow as a program grows.

~5 min

Reusable Components

A Frame that bundles several widgets.

```
class LabeledEntry(tk.Frame):
    def __init__(self, master, label):
        super().__init__(master)
        tk.Label(self, text=label).pack()
        self.e = tk.Entry(self)
        self.e.pack()
    def get(self):
        return self.e.get()
```

Result

```
A label above a text box,
and get() returns the text
```

Reusable component

A Frame subclass bundles several widgets, so one line can add the whole group anywhere.

A clean interface

Offer methods such as get(), and keep the inner widgets out of the rest of the program.

~4 min

Extra Windows With Toplevel

Open a second window on demand.

```
def popup():
    win = tk.Toplevel(root)
    win.title("Info")
    tk.Label(win, text="Popup").pack()
tk.Button(root, text="Open",
          command=popup).pack()
```

Result

```
Clicking Open shows a second
window titled Info
```

Toplevel

A Toplevel is an extra window with its own title and widgets, linked to the main window.

Close with destroy

win.destroy() closes just that window. root.destroy() ends the whole program.

~4 min

File Dialogs

Let the user choose a file.

```
from tkinter import filedialog
def pick():
    path = filedialog.askopenfilename()
    if path:
        print(path)
tk.Button(root, text="Open",
          command=pick).pack()
```

Result

```
Opens a file chooser, then prints the path
```

filedialog

askopenfilename, asksaveasfilename and askdirectory open the standard chooser windows.

Empty result

If the user cancels, the result is empty, so test it before using the path.

~4 min

Timers With after()

Repeat work without blocking.

```
def tick():
    n.set(n.get() + 1)
    root.after(1000, tick)
n = tk.IntVar()
tk.Label(root, textvariable=n).pack()
tick()
root.mainloop()
```

Result

```
A number that counts up every second
```

after(ms, func)

Schedules a function to run once after a delay. Calling after again from inside makes a repeating timer.

Never sleep

time.sleep() inside a callback freezes the window, because the event loop cannot run.

~5 min

Keeping the Window Responsive

Long jobs and the event loop.

Problem	Why	Fix
A long loop in a callback	The event loop is blocked	Split the work into steps with after()
A slow download or calculation	The window freezes while it runs	Run it in a thread
Sending results back	tkinter is not thread-safe	Put results in a Queue
Showing the results	Only the main thread should touch widgets	Poll the Queue with after()

Call tkinter from the main thread only, and let worker threads talk through a Queue.

~4 min

Organising GUI Code

Habits that scale from a script to an application.

1

Separate logic

Keep calculations apart from widgets, so you can test them.

model view

2

One class each

Give each window or component its own class.

class Window

3

Short callbacks

A handler should call a method, not hold the whole job.

command=self.save

A GUI is easier to change when the layout code and the logic do not depend on each other.

~3 min

GUI Techniques Cheat Sheet

Match the goal to the technique.

Goal	Technique
Reuse a group of widgets	A Frame subclass
Keep state	Instance attributes and bound methods
Pass an argument to a callback	partial or a lambda default
Run something repeatedly	after()
Keep the window responsive	A thread plus a Queue

> Pick the simplest technique that fits, and add structure when the program grows.

ONE-LINE TAKEAWAY

Organise a GUI into classes and components, pass data with methods, and never block the event loop.

class partial after Queue

PYTHON FOUNDATIONS · 50 MIN

Customising Widgets

Look
Fonts and colours
`font fg bg`

Shape
Borders and size
`relief padx`

Images
Pictures on widgets
`PhotoImage`

Themes
Style many at once
`ttk.Style`

~3 min

Four Ways to Change How Widgets Look

From one widget to a whole application.

1

Look

Choose fonts, colours and pictures.

font bg

2

Shape

Set borders, padding and size.

relief bd

3

Behaviour

Change options later, and react to the mouse.

config bind

4

Themes

Style many widgets with one rule.

option_add

> Every widget option can be set when the widget is created, or changed later.

~4 min

Fonts and Colours

Style text with font, fg and bg.

```
lab = tk.Label(root, text="Hello",
               font=("Arial", 20, "bold"),
               fg="white", bg="navy")
lab.pack(padx=20, pady=20)
```

Result

```
White bold Hello on a navy background
```

The font tuple

A font is a tuple of family, size and style, such as ('Arial', 20, 'bold').

Colours

Use names such as 'navy', or hex strings such as '#003366', for fg and bg.

~4 min

Borders and Relief

Give widgets edges you can see.

```
for r in ["flat", "raised", "sunken",
          "groove", "ridge"]:
    tk.Label(root, text=r, relief=r,
             bd=3, width=10).pack(pady=2)
```

Result

```
Five labels, each with a different
border style
```

relief

Sets the 3-D border style: flat, raised, sunken, groove, ridge or solid.

bd and width

bd is the border thickness in pixels. width sets the size in text characters.

~4 min

Padding and Size

Inside space and outside space are different.

Option	Where it is used	Effect
padx, pady	In the widget call	Space inside the widget, around its content
padx, pady	In pack() or grid()	Space outside the widget, in the layout
ipadx, ipady	In pack() or grid()	Extra space inside the widget
width, height	In the widget call	Size in characters for text, or pixels for images

Padding given to the widget goes inside it, and padding given to the layout call goes outside it.

~4 min

Changing Options Later

config, cget and indexing.

```
b = tk.Button(root, text="Save")
b.pack()
b.config(text="Saved", state="disabled")
print(b.cget("text"))
print(b["state"])
```

Output

```
Saved
disabled
```

config

config(option=value) changes options after the widget exists, so a program can update its look.

cget and indexing

b.cget('text') or b['text'] reads an option back.

~4 min

Showing Images

Pictures on labels and buttons.

```
img = tk.PhotoImage(file="logo.png")
lab = tk.Label(root, image=img)
lab.image = img
lab.pack()
```

Result

```
A label that shows logo.png
```

PhotoImage

PhotoImage loads PNG and GIF files. Other formats need a library such as Pillow.

Keep a reference

If the image object is garbage collected the picture disappears, so store it, as in lab.image = img.

~5 min

Custom Widget Classes

Bake a style into your own widget.

```
class FancyButton(tk.Button):
    def __init__(self, master, **kw):
        kw.setdefault("bg", "navy")
        kw.setdefault("fg", "white")
        kw.setdefault("font", ("Arial", 12))
        super().__init__(master, **kw)
FancyButton(root, text="OK").pack()
```

Result

```
A navy OK button with white text
```

Default options

setdefault gives a style to every instance, and callers can still override it.

**kw

Passing extra options through **kw keeps the subclass flexible.

~4 min

ttk Themed Widgets

A second widget set that follows a theme.

Widget	ttk class	Note
Button	ttk.Button	Styled by the theme
Entry	ttk.Entry	Text input with a themed look
Combobox	ttk.Combobox	A drop-down list, found only in ttk
Notebook	ttk.Notebook	Tabbed pages
Progressbar	ttk.Progressbar	Shows how far a task has got

ttk widgets take their look from a theme, so most of them ignore options such as bg and fg.

~5 min

Styling ttk Widgets

Choose a theme and define styles.

```
from tkinter import ttk
style = ttk.Style()
style.theme_use("clam")
style.configure("Big.TButton",
                font=("Arial", 16))
ttk.Button(root, text="OK",
           style="Big.TButton").pack()
```

Result

A large OK button in the clam theme

Themes

theme_use picks a theme, and style.theme_names() lists the available ones.

Named styles

A style name such as Big.TButton ends in the widget class, and inherits from TButton.

~4 min

The Option Database

Set defaults for whole classes of widgets.

```
root.option_add("*Font", "Arial 14")
root.option_add("*Label.foreground", "navy")
tk.Label(root, text="Styled").pack()
tk.Button(root, text="Also").pack()
```

Result

```
Both use Arial 14, and the label
text is navy
```

Option database

option_add sets defaults for a class of widgets, so you avoid repeating options.

Patterns

'*Font' means every widget, while '*Label.foreground' affects only Labels.

~4 min

Resizable Layouts

Let widgets grow with the window.

```
root.columnconfigure(0, weight=1)
root.rowconfigure(0, weight=1)
text = tk.Text(root)
text.grid(row=0, column=0, sticky="nsew")
```

Result

```
The text box grows when the window is resized
```

weight

A row or column with weight above 0 receives extra space when the window grows.

sticky

sticky='nsew' stretches the widget to fill its cell in all four directions.

~3 min

Customising Cheat Sheet

Match the goal to the option or tool.

Goal	Use
Change colours	fg and bg
Change the font	font=('Arial', 14, 'bold')
Border style	relief and bd
Show a picture	PhotoImage
Style many widgets at once	option_add or ttk.Style

Platform note: some native buttons, such as the macOS Button, may ignore bg. Use a Label or ttk styles there.

ONE-LINE TAKEAWAY

Set options when you create a widget, change them with config, and share styles through classes and themes.

font relief config ttk.Style

INTERNET PROGRAMMING · 50 MIN

Network Scripting

Sockets
TCP and UDP basics
`socket`

Servers
Handle many clients
`bind accept`

Web
Pages and APIs
`requests`

Services
Send email
`smtplib`

~3 min

Four Ways Python Talks to the Network

From a single socket to real internet services.

1

Sockets

Open connections and swap raw bytes.

socket

2

Servers

Wait for clients and answer many at once.

bind accept

3

Web

Fetch pages and call web APIs.

urllib requests

4

Services

Send email through a mail server.

smtplib

> Every network program is a client that asks, a server that answers, or both.

~4 min

Sockets and Addresses

Create a socket and say where it should connect.

```
import socket
ip = socket.gethostbyname("localhost")
print(ip)
s = socket.socket(socket.AF_INET,
                  socket.SOCK_STREAM)
```

Result

```
127.0.0.1
```

AF_INET

Selects IPv4 addresses. An address is a tuple of host and port, such as ('127.0.0.1', 5000).

SOCK_STREAM

Selects a TCP stream. SOCK_DGRAM selects UDP. The defaults are IPv4 and TCP.

~5 min

A TCP Server

Bind, listen, accept, then talk.

```
import socket
srv = socket.socket()
srv.bind(("127.0.0.1", 5000))
srv.listen(1)
conn, addr = srv.accept()
data = conn.recv(1024)
conn.sendall(data.upper())
conn.close()
```

Result

```
A client that sends hello gets HELLO back
```

bind and listen

bind attaches the socket to an address and port. listen starts queuing incoming clients.

accept

Blocks until a client connects, then returns a new socket, conn, and the client's address.

~4 min

A TCP Client

Connect, send, receive.

```
import socket
c = socket.socket()
c.connect(("127.0.0.1", 5000))
c.sendall(b"hello")
print(c.recv(1024))
c.close()
```

Output

```
b'HELLO'
```

connect

Opens a connection to the server's host and port. The server must already be listening.

Bytes, not strings

Sockets carry bytes. Use b'hi' or text.encode() to send, and data.decode() to read.

~4 min

TCP or UDP

Pick the protocol that matches the job.

Feature	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Connection	Connect first, then talk	None, just send datagrams
Delivery	Reliable and in order	May be lost or reordered
Main calls	send and recv	sendto and recvfrom
Typical use	Web, email and file transfer	DNS, video calls and games

Choose TCP when every byte must arrive, and UDP when speed matters more than perfection.

~4 min

UDP Datagrams

No connection, just messages.

```
import socket
s = socket.socket(socket.AF_INET,
                  socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 6000))
data, addr = s.recvfrom(1024)
print(data, addr)
```

Output

```
b'ping' ('127.0.0.1', 51234)
```

recvfrom

Returns the data and the sender's address, because there is no connection to remember it.

sendto

The sender names the destination each time: s.sendto(b'ping', ('127.0.0.1', 6000)).

~4 min

Timeouts and Errors

Networks fail, so plan for it.

```
c = socket.socket()
c.settimeout(3)
try:
    c.connect(("example.com", 80))
except socket.timeout:
    print("Too slow")
except OSError as e:
    print("Failed:", e)
```

Result

```
Too slow, if nothing replies within 3 seconds
```

settimeout

Limits how long connect and recv may block, in seconds. Without it a program can wait forever.

OSError

Refused or unreachable connections raise OSError or one of its subclasses.

~5 min

Serving Many Clients

One thread for each client.

```
import threading
def handle(conn):
    conn.sendall(conn.recv(1024))
    conn.close()
while True:
    conn, addr = srv.accept()
    t = threading.Thread(target=handle,
                         args=(conn,))
    t.start()
```

Result

```
Each client is served in its own thread
```

One thread each

Without threads, one slow client blocks everyone else waiting to connect.

socketserver

ThreadingTCPServer does this for you and calls a handler class for each client.

~4 min

HTTP with urllib

Fetch a web page with the standard library.

```
from urllib.request import urlopen
with urlopen("https://example.com") as r:
    print(r.status)
    html = r.read().decode()
print(html[:15])
```

Output

```
200
<!doctype html>
```

urlopen

Fetches a URL and returns a response with a status code, headers and a body.

decode

read() returns bytes, and decode() turns them into text you can search and print.

~4 min

The requests Library

A friendlier way to call web servers.

```
import requests
r = requests.get("https://api.github.com",
                 timeout=5)
print(r.status_code)
print(r.headers["content-type"])
```

Output

```
200
application/json; charset=utf-8
```

requests

A third-party library: pip install requests. It also offers post, put and delete.

json() and timeout

r.json() parses a JSON reply into dicts and lists. Always pass timeout so calls cannot hang.

~4 min

Sending Email

Build a message and hand it to a mail server.

```
import smtplib
from email.message import EmailMessage
m = EmailMessage()
m["From"] = "me@example.com"
m["To"] = "you@example.com"
m["Subject"] = "Hello"
m.set_content("Hi from Python")
with smtplib.SMTP("localhost") as s:
    s.send_message(m)
```

Result

```
The message is handed to the mail server
```

EmailMessage

Builds the headers and body, so Python handles the formatting rules for you.

SMTP details

Real servers usually need starttls() and login(user, password), often on port 587.

~3 min

Network Scripting Cheat Sheet

Match the goal to the call.

Goal	Use
Connect to a server	s.connect((host, port))
Accept clients	bind, listen, accept
Send datagrams	sendto and recvfrom
Avoid hanging	settimeout(seconds)
Fetch a page or API	requests.get(url, timeout=5)
Send email	smtplib.SMTP and send_message

Tip: ports below 1024 need administrator rights, so use 5000 or higher while practising.

ONE-LINE TAKEAWAY

A network script opens a socket, sends and receives bytes, and leans on libraries for the web and email.

socket bind recvfrom requests

INTERNET PROGRAMMING · 50 MIN

Client-Side Scripting

JavaScript
Runs in the page
`script defer`

The DOM
Find and change elements
`querySelector`

Events
React to the visitor
`addEventListener`

Forms
Check input early
`preventDefault`

~3 min

Four Skills of Client-Side Scripting

From a first script to a page that reacts.

1

Script

Run JavaScript in the browser.

script defer

2

DOM

Find and change page elements.

querySelector

3

Events

React to clicks and typing.

addEventListener

4

Forms

Check input before it is sent.

validity

Client-side code runs in the visitor's browser, so a page can react without waiting for the server.

~4 min

Adding JavaScript to a Page

Put a script in the page and run it.

```
<h1 id="msg">Hello</h1>
<script>
  const h = document.getElementById("msg");
  h.textContent = "Hello, JavaScript";
</script>
```

Result

```
The heading now reads Hello, JavaScript
```

The script tag

Inline code goes between script tags. External code uses <script src='app.js' defer>.

defer

Waits until the page is parsed, so the script can find the elements it needs.

~4 min

Variables and Functions

let, const and function.

```
const price = 20;
let qty = 3;
function total(a, b) {
  return a * b;
}
console.log(total(price, qty));
```

Output

```
60
```

const and let

const cannot be reassigned, while let can. Avoid var in new code.

Functions

Declare with function, or write an arrow function such as (a, b) ⇒ a * b.

~4 min

Objects and Arrays

Group values together.

```
const user = { name: "Asha", age: 21 };
const tags = ["web", "js"];
tags.push("dom");
console.log(user.name, tags.length);
```

Output

```
Asha 3
```

Objects

Store named values as key and value pairs, read with user.name or user['name'].

Arrays

Ordered lists with push, pop, map and filter. Indexes start at 0.

~4 min

Client Side or Server Side

Know where your code runs.

Feature	Client side	Server side
Runs in	The visitor's browser	The web server
Typical tools	JavaScript, HTML and CSS	Python, PHP, Java or Node.js
Can reach	The page and the browser	Databases and files
Trust level	Visible to users, so never trusted	Hidden from users

> Never rely on client-side checks alone. Always validate again on the server.

~4 min

Selecting Elements

Find parts of the page with CSS selectors.

```
const box = document.querySelector("#box");
const items = document.querySelectorAll("li");
console.log(items.length);
box.style.color = "teal";
```

Output

```
3, if the page has three list items
```

querySelector

Returns the first element matching a CSS selector such as #box or .card.

querySelectorAll

Returns every match as a list you can loop over with forEach.

~5 min

Changing the Page

Edit text, classes and elements.

```
const p = document.querySelector("p");
p.textContent = "Updated";
p.classList.add("highlight");
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").append(li);
```

Result

The paragraph changes and a new list item appears

textContent

Sets plain text safely. innerHTML parses markup, so avoid it with user input.

createElement

Builds a new element in memory, and append adds it to the page.

~5 min

Handling Events

Run code when something happens.

```
const btn = document.querySelector("button");
btn.addEventListener("click", (e) => {
  console.log("Clicked", e.target.tagName);
});
```

Output

```
Clicked BUTTON, each time the button is pressed
```

addEventListener

Attaches a handler to an event such as click, input, keydown or submit.

The event object

e carries details, for example e.target, the element that fired the event.

~5 min

Form Validation

Check input before it is sent.

```
const f = document.querySelector("form");
f.addEventListener("submit", (e) => {
  const email = f.email.value.trim();
  if (!email.includes("@")) {
    e.preventDefault();
    alert("Enter a valid email");
  }
});
```

Result

The form is blocked until the email contains @

preventDefault

Stops the browser from submitting the form, so you can show an error first.

HTML5 checks

Attributes such as required, type='email' and pattern validate with no script.

~4 min

Timers and Local Storage

Delay work and remember small values.

```
setTimeout(() => console.log("Hi"), 1000);
localStorage.setItem("theme", "dark");
const t = localStorage.getItem("theme");
console.log(t);
```

Output

```
dark
Hi (after 1 second)
```

setTimeout

Runs a function once after a delay in milliseconds. setInterval repeats it.

localStorage

Saves strings in the browser, and they stay after the page is closed.

~5 min

Fetching Data

Talk to a server without reloading the page.

```
async function load() {
  const r = await fetch("/api/users");
  const data = await r.json();
  console.log(data.length);
}
load();
```

Result

```
Prints how many users the server returned
```

fetch

Requests data from a URL without reloading the page. It returns a promise.

await

Pauses an async function until the promise settles. Use try and catch for errors.

~3 min

Client-Side Cheat Sheet

Match the goal to the call.

Goal	Use
Run a script after the page loads	<script src='app.js' defer>
Find one element	document.querySelector(selector)
Change text	element.textContent = 'text'
React to a click	addEventListener('click', fn)
Stop a form submitting	e.preventDefault()
Load data without a reload	await fetch(url)

Tip: press F12 to open developer tools and see console.log output and errors.

ONE-LINE TAKEAWAY

Client-side scripts run in the browser, find elements in the DOM, and change the page in response to events.

querySelector | addEventListener | preventDefault | fetch

INTERNET PROGRAMMING · 50 MIN

PyMailGUI Client

Fetch
Read mail over POP
`poplib`

Send
Deliver mail over SMTP
`smtplib`

Parse
Headers and attachments
`email`

Window
Tkinter and threads
`tkinter`

~3 min

Four Parts of a Mail Client

What a program like PyMailGUI has to do.

1

Fetch

Download messages from the server.

poplib

2

Send

Compose and deliver new mail.

smtplib

3

Parse

Split messages into parts.

email

4

Display

Show lists and messages in a GUI.

tkinter queue

PyMailGUI is a Tkinter program built on the standard poplib, smtplib and email modules.

~4 min

POP, SMTP and IMAP

Three protocols, three Python modules.

Protocol	Job	Module	Usual port
POP3	Download mail from the server	poplib	110, or 995 over SSL
SMTP	Send mail to the server	smtplib	25, or 587 with starttls
IMAP	Manage mail left on the server	imaplib	143, or 993 over SSL

PyMailGUI reads mail with POP and sends it with SMTP.

~4 min

Server Settings

Keep settings in one small module.

```
# mailconfig.py
popservername = "pop.example.com"
smtpservername = "smtp.example.com"
popusername = "me@example.com"
fetchlimit = 25
```

Used as

```
import mailconfig
```

mailconfig.py

A plain module of settings, so users can change servers without touching program code.

Passwords

Never store them in code. Ask the user in a dialog and keep them only in memory.

~4 min

Logging In with POP

Connect and count the waiting messages.

```
import poplib
srv = poplib.POP3_SSL("pop.example.com")
srv.user("me@example.com")
srv.pass_(password)
count, size = srv.stat()
print(count)
```

Output

```
12, the number of messages waiting
```

stat and list

stat returns the message count and mailbox size. list shows the size of each message.

pass_

The method name has a trailing underscore because pass is a Python keyword.

~5 min

Downloading a Message

Fetch the text and turn it into an object.

```
import email
resp, lines, octets = srv.retr(1)
raw = b"\n".join(lines)
msg = email.message_from_bytes(raw)
print(msg["Subject"])
srv.quit()
```

Output

```
Hello from Python
```

retr

Downloads one full message as a list of byte lines. top fetches only the headers and a few lines.

Headers first

Loading headers alone keeps a big mailbox fast. Full text arrives when the user opens a message.

~5 min

Walking the Message Parts

Find the text and the attachments.

```
for part in msg.walk():
    if part.is_multipart():
        continue
    name = part.get_filename()
    print(part.get_content_type(), name)
```

Output

```
text/plain None
image/png photo.png
```

walk

Visits the message and every nested part, one by one.

get_filename

Returns an attachment's file name, or None for the plain message text.

~4 min

Saving an Attachment

Decode the payload and write it to disk.

```
data = part.get_payload(decode=True)
with open(name, "wb") as f:
    f.write(data)
```

Result

```
The attachment is saved as photo.png
```

decode=True

Decodes base64 or quoted-printable text back into the original bytes.

Safe names

Senders choose file names, so clean them and pick the folder yourself.

~5 min

Composing a Message

Headers, text and an attachment.

```
from email.message import EmailMessage
m = EmailMessage()
m["From"] = "me@example.com"
m["To"] = "you@example.com"
m["Subject"] = "Report"
m.set_content("See attached.")
m.add_attachment(data, maintype="application",
                 subtype="pdf", filename="r.pdf")
```

Result

```
A multipart message with text and one PDF
```

EmailMessage

Sets headers like a dictionary, and builds a multipart message when you add an attachment.

add_attachment

Takes bytes, a media type and a file name, and encodes them for sending.

~4 min

Sending with SMTP

Encrypt, log in, deliver.

```
with smtplib.SMTP("smtp.example.com", 587) as s:
    s.starttls()
    s.login(user, password)
    s.send_message(m)
```

Result

```
The server accepts the message for delivery
```

starttls

Switches the connection to encrypted mode before the password is sent.

Reply and forward

A reply addresses the sender and adds Re: to the subject. A forward adds Fwd: and the old text.

~5 min

The Message List Window

One row for each message.

```
import tkinter as tk
root = tk.Tk()
lb = tk.Listbox(root, width=60)
lb.pack(fill="both", expand=True)
for subj in subjects:
    lb.insert("end", subj)
lb.bind("<Double-Button-1>", open_message)
root.mainloop()
```

Result

```
Double-clicking a subject opens that message
```

Listbox

Shows one line per message, such as the sender and subject, ready to click.

Several windows

PyMailGUI keeps separate windows for the list, each message read, and each message written.

~5 min

Keeping the GUI Responsive

Slow network work belongs in a thread.

```
import threading, queue
q = queue.Queue()
def worker():
    q.put(fetch_headers())
threading.Thread(target=worker).start()
def poll():
    if not q.empty():
        show(q.get())
    root.after(100, poll)
```

Result

```
The window stays responsive while mail downloads
```

Worker thread

Slow network calls run in a thread, so the window never freezes.

queue and after

The thread posts results to a queue, and the GUI reads it with after. Tkinter is not thread-safe.

~3 min

PyMailGUI Cheat Sheet

Match the goal to the call.

Goal	Use
Log in and count mail	POP3_SSL, user, pass_, stat
Download one message	retr(n), then message_from_bytes
Find attachments	msg.walk() and get_filename()
Build a new message	EmailMessage and add_attachment
Send it	SMTP, starttls, login, send_message
Keep the window alive	thread, queue and root.after

Tip: test with a throwaway mailbox, and never put real passwords in source files.

ONE-LINE TAKEAWAY

A mail client fetches with poplib, parses with email, sends with smtplib, and uses threads to stay responsive.

poplib email smtplib queue

INTERNET PROGRAMMING · 50 MIN

Server-Side Scripting

Requests
GET and POST data
`request`

Routes
Map URLs to code
`@app.route`

Templates
Build HTML pages
`render_template`

Data
Databases and sessions
`sqlite3 session`

~3 min

Four Jobs of a Server-Side Script

What happens between a click and a page.

1

Receive

Read the request from the browser.

request

2

Process

Run logic and check the input.

validate

3

Store

Save and load data safely.

sqlite3

4

Respond

Send back HTML or JSON.

render_template

> The browser asks, the server script runs, and only the result travels back to the visitor.

~4 min

A First Server-Side Page

A web server in six lines.

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "<h1>Hello, visitor</h1>"
app.run()
```

Result

```
Browsing to http://127.0.0.1:5000 shows the heading
```

Flask

A small framework installed with pip install flask. It handles HTTP so you write only the logic.

route

The decorator links a URL path to a Python function, called each time that URL is requested.

~4 min

Routes and URL Parameters

Pull values out of the path.

```
@app.route("/item/<int:item_id>")
def item(item_id):
    return f"Item number {item_id}"
@app.route("/add/<int:a>/<int:b>")
def add(a, b):
    return str(a + b)
```

Output

```
/item/7 shows Item number 7
/add/2/3 shows 5
```

Converters

int, float and path convert the part of the URL before your function sees it.

Why int

A converter rejects other values with a 404, which keeps bad input out of your code.

~5 min

Reading Form Data

GET reads pages, POST sends data.

```
from flask import request, redirect
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = request.form["user"].strip()
        return redirect("/welcome")
    return "Show the login form"
```

Result

```
A POST to /login redirects the browser to /welcome
```

request.form

Holds POST fields by name. request.args holds query string values such as ?page=2.

GET or POST

Use GET to read pages, and POST to send data that changes something on the server.

~4 min

Common Status Codes

Every response starts with a number.

Code	Meaning	When you see it
200 OK	The request succeeded	A normal page or JSON reply
302 Found	Go to another URL	After a form is saved
404 Not Found	No such page	An unknown or mistyped URL
500 Server Error	The script crashed	A bug in the server code

The first digit gives the class: 2 success, 3 redirect, 4 client error, 5 server error.

~5 min

Templates

Fill an HTML page with data.

```
from flask import render_template
@app.route("/hello/<name>")
def hello(name):
    return render_template("hello.html",
                           name=name)
```

templates/hello.html

```
<h1>Hello {{ name }}</h1>
```

Jinja2 templates

Placeholders such as {{ name }} are filled in by the server before the page is sent.

Auto-escaping

Special characters in values are escaped, which blocks most script injection.

~5 min

Validating Input

Never trust what the browser sends.

```
age = request.form.get("age", "")
if not age.isdigit():
    return "Age must be a number", 400
age = int(age)
if age > 120:
    return "Age is too large", 400
```

Result

```
Bad input gets a 400 reply and stops there
```

Check on the server

Verify type, length and range here, even if JavaScript already checked in the browser.

400 status

Return 400 Bad Request when the client sent something unusable.

~5 min

Storing Data with SQLite

A database in a single file.

```
import sqlite3
db = sqlite3.connect("app.db")
db.execute("CREATE TABLE IF NOT EXISTS notes"
           "(id INTEGER PRIMARY KEY, text TEXT)")
db.execute("INSERT INTO notes(text) VALUES (?)",
           (txt,))
db.commit()
rows = db.execute("SELECT text FROM notes")
```

Result

```
Each saved note can be read back from app.db
```

Placeholders

The ? marks are filled in by the driver, which stops SQL injection. Never build SQL with f-strings.

commit

Changes reach the file only after commit(). Without it they are lost.

~5 min

Sessions and Cookies

Remember a visitor between requests.

```
from flask import session
app.secret_key = "change-me"
@app.route("/login", methods=["POST"])
def login():
    session["user"] = request.form["user"]
    return redirect("/me")
@app.route("/me")
def me():
    return "yes" if "user" in session else "no"
```

Result

```
After login, /me answers yes
```

session

A signed cookie holding per-visitor data. Changes are detected, but visitors can read it.

secret_key

Signs the cookie. Keep it private and random, and keep it out of version control.

~4 min

Returning JSON

Serve data for client-side code.

```
from flask import jsonify
@app.route("/api/notes")
def notes():
    rows = db.execute("SELECT text FROM notes")
    return jsonify([r[0] for r in rows])
```

Output

```
["buy milk", "call Asha"]
```

jsonify

Turns Python lists and dicts into JSON and sets the Content-Type header for you.

Client meets server

Browser code can call this route with fetch and read the reply with r.json().

~4 min

Error Pages and Debug Mode

Plan for failure.

```
@app.errorhandler(404)
def missing(e):
    return "Page not found", 404
app.run(debug=False)
```

Result

```
Unknown URLs show a friendly message
```

errorhandler

Runs when a given status code occurs, so you control what visitors see.

debug mode

debug=True shows detailed errors in the browser. Never enable it on a public site.

~3 min

Server-Side Cheat Sheet

Match the goal to the call.

Goal	Use
Serve a page	@app.route("/") above a function
Read form fields	request.form["name"]
Build HTML	render_template("page.html", x=x)
Store data safely	sqlite3 with ? placeholders
Remember a visitor	session["user"] = value
Return JSON	jsonify(data)

Tip: run python app.py, then open http://127.0.0.1:5000 in your browser.

ONE-LINE TAKEAWAY

A server-side script reads the request, validates input, talks to the database, and sends back a page or JSON.

route

request.form

sqlite3

session

INTERNET PROGRAMMING · 50 MIN

PyMailGUI Server

SMTP
Receive mail from clients
`aiosmtpd`

POP3
Serve mail on request
`socketserver`

Mailbox
Store messages on disk
`mailbox`

Safety
Logins and encryption
`hashlib ssl`

~3 min

Four Jobs of a Mail Server

What sits on the other end of the connection.

1

Receive

Accept mail from clients.

SMTP

2

Store

Keep messages for each user.

mailbox

3

Serve

Hand mail over when asked.

POP3

4

Protect

Check logins and encrypt.

hashlib ssl

PyMailGUI is only the client. These server pieces answer its smtplib and poplib calls.

~4 min

Mail Server Commands

Both protocols are plain text.

Protocol	Main commands	Reply style
SMTP	EHLO, MAIL FROM, RCPT TO, DATA, QUIT	Numeric codes such as 250
POP3	USER, PASS, STAT, RETR, DELE, QUIT	+OK or -ERR
Both	One command on each line	One reply for each command

Because they are plain text, you can test both with a socket or a telnet session.

~5 min

An SMTP Conversation

What smtplib says to the server.

```
EHLO laptop
MAIL FROM:<me@example.com>
RCPT TO:<you@example.com>
DATA
Subject: Hello

Hello there
.
QUIT
```

Server replies

```
250 OK to each command, 354 after DATA
250 after the final dot, 221 after QUIT
```

Envelope and body

MAIL FROM and RCPT TO name the sender and recipients. DATA carries the headers and text.

The lone dot

A line holding only a full stop ends the message body.

~5 min

Receiving Mail in Python

A small SMTP server with aiosmtpd.

```
from aiosmtpd.controller import Controller
class Handler:
    async def handle_DATA(self, server, session,
                          envelope):
        print(envelope.mail_from, envelope.rcpt_tos)
        return "250 Message accepted"
ctl = Controller(Handler(), port=8025)
ctl.start()
```

Result

```
Prints sender and recipients for each message
```

aiosmtpd

A third-party SMTP server library. The old smtpd module was removed in Python 3.12.

handle_DATA

Called once for each message, with the sender, recipients and full content in envelope.

~4 min

Storing Mail with mailbox

One folder for each user.

```
import mailbox
box = mailbox.Maildir("asha", create=True)
key = box.add(msg)
print(len(box))
for m in box:
    print(m["Subject"])
```

Output

```
1
Hello
```

Maildir

One file per message in a folder, so adding mail never rewrites a big file.

mbox

mailbox.mbox keeps all messages in one file. It is simple, but needs file locking.

~5 min

A Toy POP3 Server

Greet the client and answer commands.

```
import socketserver
class Pop(socketserver.StreamRequestHandler):
    def handle(self):
        self.wfile.write(b"+OK ready\r\n")
        for line in self.rfile:
            if line.strip().upper() == b"QUIT":
                self.wfile.write(b"+OK bye\r\n")
                break
            self.wfile.write(b"-ERR unknown\r\n")
```

Result

```
Greets, replies +OK bye to QUIT, else -ERR
```

StreamRequestHandler

Gives self.rfile and self.wfile, so a socket reads and writes like a file.

CRLF endings

POP3 lines end with \r\n, and every reply begins with +OK or -ERR.

~5 min

Serving Messages

Answer STAT and RETR.

```
elif cmd == b"STAT":
    self.reply(f"+OK {len(msgs)} {size}")
elif cmd == b"RETR":
    n = int(arg) - 1
    self.reply("+OK message follows")
    self.wfile.write(msgs[n] + b"\r\n.\r\n")
```

Result

```
STAT reports count and size, RETR sends one
```

The closing dot

A multi-line reply ends with a line holding only a dot. Lines that start with a dot get an extra dot.

The msgs list

Here msgs holds each message as bytes. A real server reads them from a mailbox.

~4 min

Serving Many Clients

One thread for each connection.

```
Server = socketserver.ThreadingTCPServer
Server.daemon_threads = True
with Server(("127.0.0.1", 8110), Pop) as srv:
    srv.serve_forever()
```

Result

```
Each connection is handled in its own thread
```

ThreadingTCPServer

Starts a new thread for every client, so one slow client does not block the rest.

Port 8110

Real POP3 uses port 110, which needs administrator rights. Use a high port while testing.

~5 min

Checking Logins

Store hashes, not passwords.

```
import hashlib, hmac, os
salt = os.urandom(16)
h = hashlib.pbkdf2_hmac("sha256", pw,
                        salt, 200_000)
ok = hmac.compare_digest(h, stored)
```

Result

```
ok is True only when the password matches
```

Salted hash

Store a salted hash, never the password itself, so a stolen file does not reveal passwords.

compare_digest

Compares in constant time, which avoids leaking clues through timing.

~5 min

Encrypting with TLS

Wrap each connection in SSL.

```
import ssl
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain("cert.pem", "key.pem")
secure = ctx.wrap_socket(conn, server_side=True)
secure.sendall(b"+OK ready\r\n")
```

Result

```
Traffic between client and server is encrypted
```

Certificates

A certificate proves the server's identity. Self-signed ones are fine for testing only.

SSL or STARTTLS

POP3 over SSL uses port 995. SMTP on 587 starts plain, then upgrades with STARTTLS.

~4 min

Testing with a Client

Point a client at your own server.

```
import smtplib
with smtplib.SMTP("localhost", 8025) as s:
    s.sendmail("me@example.com",
               ["you@example.com"],
               "Subject: Test\r\n\r\nHello")
```

Result

```
The server prints the sender and recipient
```

Local test server

Run python -m aiosmtpd -n -l localhost:8025 to print every message it receives.

Real clients

A mail client such as PyMailGUI can be pointed at your server in its settings.

~3 min

Mail Server Cheat Sheet

Match the goal to the tool.

Goal	Use
Accept mail over SMTP	aiosmtpd Controller and handle_DATA
Store a message	mailbox.Maildir(path).add(msg)
Speak POP3	StreamRequestHandler, +OK and -ERR
Serve many clients	ThreadingTCPServer
Check passwords	pbkdf2_hmac and compare_digest
Encrypt traffic	ssl.SSLContext and wrap_socket

Tip: build a toy server to learn, but run Postfix or Dovecot in production.

ONE-LINE TAKEAWAY

A mail server takes mail by SMTP, stores it in mailboxes, serves it by POP3, and guards it with logins and TLS.

SMTP | mailbox | POP3 | ssl

TOOLS AND TECHNIQUES · 50 MIN

Databases and Persistence

Files
Text, CSV and JSON
`json csv`

Objects
Save Python objects
`pickle shelve`

SQL
Tables and queries
`sqlite3`

Mapping
Rows as objects
`dataclass`

~3 min

Four Ways to Make Data Last

From plain files to a real database.

1

Text files

Plain files you can read.

json csv

2

Pickles

Save whole Python objects.

pickle

3

Shelves

A dictionary kept on disk.

shelve

4

SQL

Tables with fast queries.

sqlite3

Persistence means data that outlives the program that created it.

~4 min

Choosing a Storage Tool

Match the tool to the data.

Tool	Best for	Watch out for
JSON or CSV	Readable data shared with other programs	Simple types only
pickle	Almost any Python object	Unsafe to load from strangers
shelve	Many objects looked up by key	Not for several writers at once
SQLite	Structured data and queries	Needs some SQL

Start with the simplest tool that fits, and move to SQL when you need queries.

~4 min

Saving JSON

Readable data that other programs understand.

```
import json
data = {"name": "Asha", "scores": [90, 85]}
with open("data.json", "w") as f:
    json.dump(data, f)
with open("data.json") as f:
    print(json.load(f)["name"])
```

Output

```
Asha
```

dump and load

dump writes to a file and load reads it back. dumps and loads work with strings.

Simple types only

JSON holds dicts, lists, strings, numbers, booleans and None. Other objects need converting.

~4 min

Reading CSV

Tables of values as rows of text.

```
import csv
with open("people.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["age"])
```

Output

```
Asha 21
```

DictReader

Reads each row as a dictionary keyed by the names in the header line.

newline=''

Pass newline='' when opening CSV files so line endings are handled correctly.

~4 min

Pickling Objects

Save almost any Python value.

```
import pickle
data = {"name": "Asha", "tags": {"a", "b"}}
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)
with open("data.pkl", "rb") as f:
    print(pickle.load(f)["name"])
```

Output

```
Asha
```

Binary mode

Pickle files hold bytes, so open them with wb to write and rb to read.

Security

Loading a pickle can run code. Only load files that you created yourself.

~4 min

Pickling Your Own Classes

Instances can be saved too.

```
class Person:
    def __init__(self, name):
        self.name = name
p = Person("Asha")
data = pickle.dumps(p)
print(pickle.loads(data).name)
```

Output

```
Asha
```

Class must be importable

Unpickling needs the class definition available under the same module name.

State only

Only the instance data is saved. Methods and code come from the class itself.

~5 min

Shelves

A dictionary that lives in a file.

```
import shelve
with shelve.open("people") as db:
    db["asha"] = {"age": 21}
    db["ravi"] = {"age": 25}
with shelve.open("people") as db:
    print(db["asha"]["age"], len(db))
```

Output

```
21 2
```

Dictionary on disk

Keys must be strings. Values can be any object that pickle can save.

Update gotcha

db['a']['age'] = 22 changes a temporary copy. Assign the whole value back instead.

~5 min

SQLite Basics

A full database in one file.

```
import sqlite3
db = sqlite3.connect("people.db")
db.execute("CREATE TABLE IF NOT EXISTS people"
           "(name TEXT, age INTEGER)")
db.execute("INSERT INTO people VALUES (?, ?)",
           ("Asha", 21))
db.commit()
```

Result

```
Creates people.db and saves one row
```

connect

Opens the database file, creating it if it does not exist yet.

commit

Saves the changes. Without it an insert is lost when the connection closes.

~5 min

Querying Rows

Ask for exactly what you need.

```
cur = db.execute("SELECT name, age FROM people"
                 " WHERE age > ?", (18,))
for name, age in cur:
    print(name, age)
```

Output

```
Asha 21
```

Placeholders

Pass values as a tuple with ? marks. Never join strings into SQL, to prevent injection.

Fetching

Loop over the cursor as shown, or call fetchone and fetchall for rows in one go.

~5 min

Transactions

Save related changes together.

```
up = "UPDATE people SET age = age + 1"
rm = "DELETE FROM people WHERE age > 100"
try:
    with db:
        db.execute(up)
        db.execute(rm)
except sqlite3.Error as e:
    print("Rolled back:", e)
```

Result

```
Both changes are saved together, or neither
```

with db

Commits when the block succeeds, and rolls back if it raises an error.

Why it matters

A transaction stops the database ever showing a half-finished update.

~5 min

Rows as Objects

Turn query results into Python objects.

```
from dataclasses import dataclass
@dataclass
class Person:
    name: str
    age: int
sql = "SELECT name, age FROM people"
people = [Person(*r) for r in db.execute(sql)]
print(people[0].name)
```

Output

```
Asha
```

dataclass

Gives a plain class an automatic constructor, so each row becomes a small object.

ORMs

Libraries such as SQLAlchemy do this mapping for you, and write the SQL as well.

~3 min

Persistence Cheat Sheet

Match the goal to the tool.

Goal	Use
Save simple data as text	json.dump and json.load
Read a table of values	csv.DictReader
Save any Python object	pickle.dump and pickle.load
Keep objects by key	shelve.open(name)
Store and query records	sqlite3 with ? placeholders
Save changes together	with db: (commit or rollback)

Tip: back up data files, and keep passwords and other secrets out of them.

ONE-LINE TAKEAWAY

Data lasts when you save it: JSON to share, pickle and shelve for objects, SQL when you need to query.

json pickle shelve sqlite3

PYTHON FOUNDATIONS · 50 MIN

Data Structures in Python

Built-in
Ways to hold data
`list dict set`

Linear
Order of access
`stack queue`

Linked
Nodes and links
`Node`

Nonlinear
Branches and networks
`tree graph`

~3 min

Four Families of Data Structures

Each one trades speed for flexibility in a different way.

1

Built-in

list, tuple, set and dict cover most needs.

dict set

2

Stacks and queues

Control which item comes out next.

deque heapq

3

Linked structures

Items joined by references.

Node

4

Trees and graphs

Data with branches and connections.

tree graph

A data structure is a way of organising data so that the operations you need are fast.

~4 min

Built-in Data Structures

Four structures, four sets of rules.

Structure	Ordered	Changeable	Duplicates
list	Yes	Yes	Allowed
tuple	Yes	No	Allowed
set	No	Yes	Not allowed
dict	By insertion	Yes	Keys must be unique

> Choose by how you will use the data: by position, by key, or by membership.

~4 min

Cost of Common Operations

Why the choice of structure matters as data grows.

Operation	list	set or dict
Find a value or key	O(n)	O(1) on average
Add at the end	O(1)	O(1) on average
Insert at the front	O(n)	Not applicable
Index by position	O(1)	Not applicable

O(n) means time grows with the number of items, and O(1) means it stays about the same.

~4 min

Sets for Membership and Maths

Unique items, and fast tests.

```
a = {1, 2, 3}
print(a | {3, 4})
print(a & {3, 4})
print(a - {3, 4})
```

Output

```
{1, 2, 3, 4}
{3}
{1, 2}
```

Set algebra

| joins, & intersects, and - subtracts. A set drops duplicates automatically.

Fast membership

x in a on a set takes constant time on average, unlike a list.

~4 min

Dictionaries as Lookup Tables

Find a value by its key.

```
words = ["a", "b", "a"]
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print(max(counts, key=counts.get))
```

Output

```
{'a': 2, 'b': 1}
a
```

get with a default

counts.get(w, 0) returns 0 for a new key, so no check is needed first.

collections.Counter

Counter(words) does this counting for you, and offers most_common().

~4 min

Stacks

Last in, first out.

```
stack = []
stack.append("a")
stack.append("b")
print(stack.pop())
print(stack)
```

Output

```
b
['a']
```

Last in, first out

A stack returns the newest item first. append pushes and pop removes the top.

Where stacks appear

Undo history, back buttons and bracket matching all use stacks.

~4 min

Queues

First in, first out.

```
from collections import deque
q = deque()
q.append("a")
q.append("b")
print(q.popleft())
print(list(q))
```

Output

```
a
['b']
```

First in, first out

A queue serves the oldest item first. append adds at the back and popleft removes from the front.

Why deque

list.pop(0) shifts every item, but deque.popleft() is fast at both ends.

~4 min

Heaps and Priority Queues

Always the smallest item next.

```
import heapq
h = [5, 1, 3]
heapq.heapify(h)
heapq.heappush(h, 0)
print(heapq.heappop(h))
print(heapq.nsmallest(2, h))
```

Output

```
0
[1, 3]
```

Priority queue

A heap gives the smallest item first, with O(log n) time for each push or pop.

Largest first

heapq is a min-heap. Push negative numbers, or use nlargest, to get the biggest.

~5 min

Linked Lists

Items joined by references.

```
class Node:
    def __init__(self, v, nxt=None):
        self.v, self.next = v, nxt
head = Node(1, Node(2, Node(3)))
n = head
while n:
    print(n.v, end=" ")
    n = n.next
```

Output

```
1 2 3
```

Nodes and links

Each node holds a value and a reference to the next node. The list is just its head.

The trade-off

Inserting after a known node is fast, but reaching the nth item means walking the chain.

~5 min

Trees and Traversal

A node whose children are trees.

```
tree = (2, (1, None, None), (3, None, None))
def inorder(t):
    if t is None:
        return []
    v, l, r = t
    return inorder(l) + [v] + inorder(r)
print(inorder(tree))
```

Output

```
[1, 2, 3]
```

Recursive shape

A tree node has children that are themselves trees, so recursion fits naturally.

Binary search tree

With smaller values on the left and larger on the right, an in-order walk lists values in sorted order.

~4 min

Graphs

Nodes and the connections between them.

```
g = {"A": ["B", "C"], "B": ["D"],
     "C": ["D"], "D": []}
for node in g:
    print(node, "->", g[node])
```

Output

```
A -> ['B', 'C']
B -> ['D']
C -> ['D']
D -> []
```

Adjacency dictionary

Each key is a node and its list holds the neighbours. This is a compact way to store a graph.

Searching graphs

Breadth-first search uses a queue. Depth-first search uses a stack or recursion.

~3 min

Choosing a Structure

Match the need to the structure.

Need	Structure
Ordered items that change	list
Fast membership tests	set
Lookup by key	dict
First in, first out	collections.deque
Always the smallest next	heapq

> Start with a list or a dict, and move to something specialised only when you need it.

ONE-LINE TAKEAWAY

Pick the structure that makes your most common operation fast, and keep the code simple.

dict deque heapq tree

DATA PREPROCESSING · 50 MIN

Data Loading and Storage

Text files
CSV and delimited
`read_csv`

Other formats
JSON and Excel
`read_json`

Binary
Compact and fast
`Parquet pickle`

Databases
Tables and queries
`read_sql`

~3 min

Four Steps From Source to Storage

Preprocessing starts with getting data in and out.

1

Read

Load text files into DataFrames, with the right options.

read_csv

2

Write

Save results as text, JSON or Excel.

to_csv

3

Store

Use binary formats when speed and types matter.

parquet pickle

4

Connect

Pull data from databases and the web.

read_sql

> Loading data well saves cleaning work later, because most problems begin at the read step.

~4 min

Reading a CSV File

Delimited text becomes a DataFrame.

```
from io import StringIO
text = "a,b\n1,2\n3,4"
df = pd.read_csv(StringIO(text))
print(df.shape)
print(df.sum().tolist())
```

Output

```
(2, 2)
[4, 6]
```

read_csv

Reads a delimited text file into a DataFrame. The first row becomes the column names by default.

A path or a buffer

It accepts a file path, a URL, or any file-like object such as StringIO.

~4 min

Common read_csv Options

Tell pandas how the file is laid out.

Option	Use it to	Example
sep	Set a different delimiter	sep=';'
header, names	Skip or supply column names	header=None, names=['x', 'y']
index_col	Use a column as the row labels	index_col='id'
skiprows	Skip lines at the top	skiprows=2
usecols	Load only some columns	usecols=['a', 'c']

header=None tells pandas the first line is data, and names then supplies the column labels.

~4 min

Dates and Data Types

Control how columns are converted.

```
text = "d,n\n2026-01-05,7\n2026-02-10,8"
df = pd.read_csv(StringIO(text),
                 parse_dates=["d"],
                 dtype={"n": "float64"})
print(df["d"].dt.month.tolist())
print(df["n"].tolist())
```

Output

```
[1, 2]
[7.0, 8.0]
```

parse_dates

Converts the listed columns to datetimes, so .dt methods such as month work.

dtype

A dictionary of dtypes avoids wrong guesses, such as an ID column read as numbers.

~4 min

Missing Value Markers

Tell pandas what counts as missing.

```
text = "a,b\n1,?\n2,5"
df = pd.read_csv(StringIO(text),
                 na_values=["?"])
print(df["b"].tolist())
print(df["b"].isna().sum())
```

Output

```
[nan, 5.0]
1
```

Default markers

NA, N/A, NaN and an empty field already count as missing.

na_values

na_values=['?'] adds extra markers, and keep_default_na=False turns the defaults off.

~4 min

Reading in Chunks

Process a file bigger than memory.

```
text = "a\n1\n2\n3\n4\n5"
total = 0
for chunk in pd.read_csv(StringIO(text),
                         chunksize=2):
    total += chunk["a"].sum()
print(total)
```

Output

```
15
```

chunksize

Reads the file in pieces of n rows, so a file bigger than memory can be processed.

Combine results

Keep a running total, or collect a small summary from each chunk, instead of keeping the chunks.

~4 min

Writing Text Files

to_csv sends a DataFrame back to text.

```
df = pd.DataFrame({'a': [1, 2],
                   'b': ['x', None]})
print(df.to_csv(index=False, na_rep='NULL'))
```

Output

```
a,b
1,x
2,NULL
```

to_csv

Writes a DataFrame as text. With no path it returns the text, which is handy for checking.

index and na_rep

index=False leaves out the row labels, and na_rep sets the text written for missing values.

~4 min

JSON Files

Lists of records and nested data.

```
df = pd.DataFrame({"a": [1, 2]})
print(df.to_json(orient="records"))
j = StringIO('[{"a": 5}]')
print(pd.read_json(j).shape)
```

Output

```
[{"a":1},{"a":2}]
(1, 1)
```

orient

orient='records' writes a list of objects, one per row. Other values include 'split' and 'index'.

read_json

Reads JSON text or files into a DataFrame. In newer pandas, wrap literal JSON text in StringIO.

~4 min

Other File Formats

Matching readers and writers.

Format	Read	Write	Needs
Excel	read_excel	to_excel	openpyxl for .xlsx
JSON	read_json	to_json	Nothing extra
HTML tables	read_html	to_html	lxml or similar
Parquet	read_parquet	to_parquet	pyarrow or fastparquet
Pickle	read_pickle	to_pickle	Nothing extra

Install the optional engine first, such as openpyxl for Excel, or pandas raises an ImportError.

~4 min

Choosing a Storage Format

Each format has a strength and a catch.

Format	Strength	Watch out for
CSV	Readable by almost anything	Slow, and dtypes are lost
Parquet	Compact, fast, keeps dtypes	Needs an engine, not readable
Pickle	Saves any Python object	Python only, and only for trusted files
SQL database	Queries and shared access	Needs a database set up

> Never unpickle data from a source you do not trust, because it can run code.

~4 min

Pickle and Parquet

Binary formats keep types exactly.

```
df = pd.DataFrame({"a": [1, 2]})
df.to_pickle("df.pkl")
back = pd.read_pickle("df.pkl")
print(back.equals(df))
df.to_parquet("df.parquet")
print(pd.read_parquet("df.parquet").shape)
```

Output

```
True
(2, 1)
```

pickle

Saves a DataFrame exactly, with dtypes and index. Only load pickles you trust.

Parquet

A compact, column-based format that keeps dtypes and reads quickly. Needs pyarrow or fastparquet.

~5 min

Working With a Database

sqlite3 ships with Python.

```
import sqlite3
con = sqlite3.connect(":memory:")
df = pd.DataFrame({"a": [1, 2, 3]})
df.to_sql("t", con, index=False)
q = "SELECT SUM(a) AS s FROM t"
print(pd.read_sql(q, con)["s"].tolist())
```

Output

```
[6]
```

to_sql and read_sql

to_sql writes a table, and read_sql runs a query and returns a DataFrame.

Connections

sqlite3 ships with Python. Other databases usually connect through SQLAlchemy.

~3 min

Loading From the Web

Common sources and the tools for them.

Source	Tool
A CSV at a URL	pd.read_csv('https://...')
JSON from an API	requests.get(url).json(), then pd.DataFrame(data)
Tables on a web page	pd.read_html(url)
An Excel file at a URL	pd.read_excel('https://...')

> Check the response first, such as its status code, before turning it into a DataFrame.

ONE-LINE TAKEAWAY

Read data with the right options, check what arrived, and store results in a format that fits the next step.

read_csv to_csv Parquet read_sql